

Structural Patterns – Part 1

Adapter · Bridge · Composite · Decorator

Sub-Chapter 02·03 | Simplest Examples | With Diagrams

Adapter	Plug incompatible interfaces together
Bridge	Separate abstraction from implementation
Composite	Treat one object and a group uniformly
Decorator	Add behaviour by wrapping, not subclassing

A Adapter Pattern

Convert an incompatible interface into the one the client expects

You have an existing class whose interface does not match what your code expects. Instead of changing either side, write an **Adapter** that wraps the old class and translates calls.

Simplest Example: OldPrinter → ModernPrinter

Client code expects ModernPrinter.print(String). You only have OldPrinter.oldPrint(String). The Adapter sits in between so neither side needs to change.

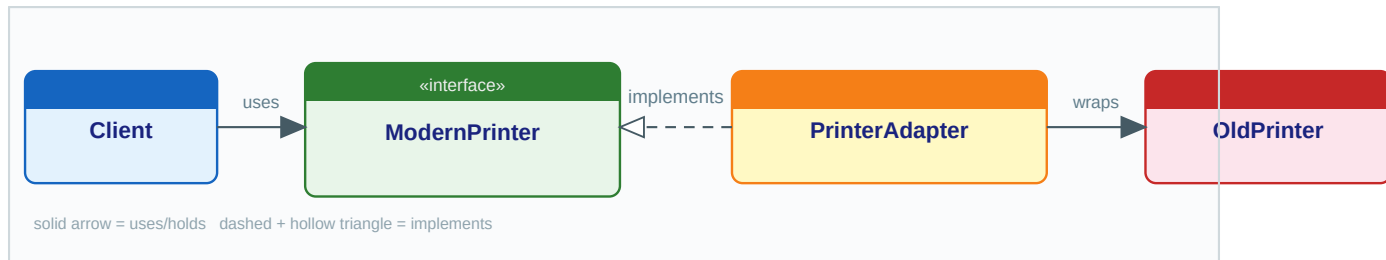


Fig 1 — Adapter structure: PrinterAdapter translates print() calls to oldPrint()

■ Without Adapter — client tightly coupled to Adaptee:

```
OldPrinter old = new OldPrinter();
old.oldPrint("Hello");    // every caller must know the old API
                          // changing printer brand = edit everywhere
```

■ With Adapter — client only sees ModernPrinter:

```
// Target — what the client expects
interface ModernPrinter { void print(String text); }

// Adaptee — existing class, cannot change
class OldPrinter {
    void oldPrint(String text) {
        System.out.println("OLD: " + text);
    }
}

// Adapter — implements Target, wraps Adaptee
class PrinterAdapter implements ModernPrinter {
    private OldPrinter old = new OldPrinter();
    public void print(String text) {
        old.oldPrint(text);    // translate the call
    }
}

// Client only ever sees ModernPrinter:
ModernPrinter p = new PrinterAdapter();
p.print("Hello");    // prints: OLD: Hello
// Swap to a new printer? Just change one line.
```

Key Rules

- Adapter implements Target interface; holds Adaptee by composition (Object Adapter).
- Client never knows an Adapter exists — it sees only the Target interface.
- Prefer Object Adapter (composition) over Class Adapter (inheritance) in Java.
- Real Java examples: InputStreamReader(InputStream), Arrays.asList(array).

Q&A;

Q1. What problem does Adapter solve?

It lets two classes with incompatible interfaces work together by inserting a translator — no changes to either class.

Q2. What are Target, Adaptee, and Adapter?

Target = interface the client expects. Adaptee = existing class with a different interface. Adapter = class that implements Target and delegates to Adaptee.

Q3. Object Adapter vs Class Adapter?

Object Adapter wraps Adaptee by composition (Java standard). Class Adapter extends Adaptee — rarely used in Java because it requires multiple class inheritance.

Q4. Give a Java standard-library example.

InputStreamReader adapts InputStream (bytes) to Reader (characters). Arrays.asList() adapts a plain array to the List interface.

Q5. Adapter vs Facade?

Facade simplifies a complex subsystem behind a new interface. Adapter makes an existing interface match another. Facade hides complexity; Adapter resolves a mismatch.

Q6. When should you NOT use Adapter?

When you can refactor the Adaptee directly. Adapters add indirection — only use them when the source is a legacy or third-party class you cannot change.

B Bridge Pattern

Decouple abstraction from implementation so both can vary independently

Without Bridge, combining two independent dimensions (remote type × device type) creates a subclass for *every combination*. Bridge separates them via a composition link so each side grows on its own.

Simplest Example: Remote Control + Device

A **RemoteControl** (abstraction) holds a **Device** (implementation). The remote delegates low-level work to Device — it does not care whether the device is a TV or a Radio.

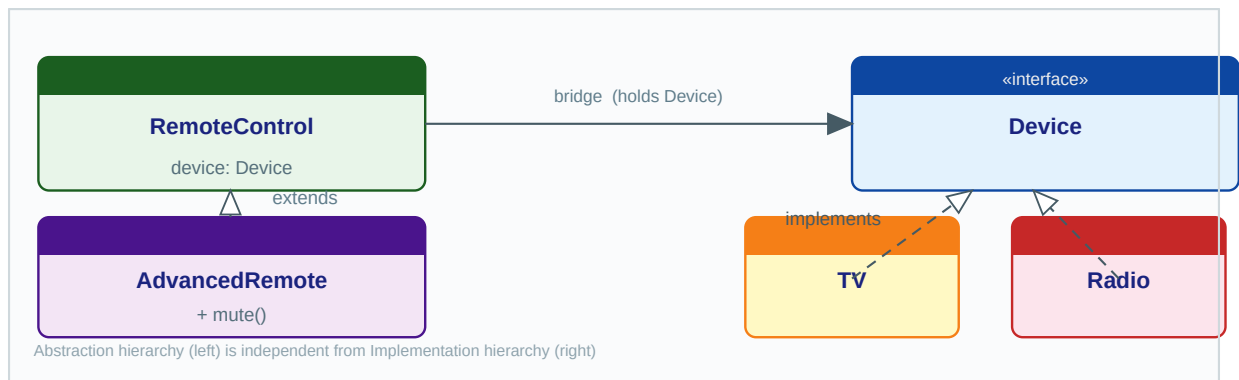


Fig 2 — Bridge structure: two independent hierarchies connected by a composition link

■ Without Bridge — one class per combination:

```

class TVRemote { void volumeUp() { /* TV logic */ } }
class RadioRemote { void volumeUp() { /* Radio logic */ } }
// Add AdvancedTVRemote + AdvancedRadioRemote -> 4 classes.
// 5 remote types x 10 devices = 50 classes.
  
```

■ With Bridge — add any remote or any device independently:

```

// ----- Implementation side (Device hierarchy) -----
interface Device {
    void volumeUp();
    void volumeDown();
    String name();
}
class TV implements Device {
    public void volumeUp() { System.out.println("TV vol +"); }
    public void volumeDown() { System.out.println("TV vol -"); }
    public String name() { return "TV"; }
}
class Radio implements Device {
    public void volumeUp() { System.out.println("Radio vol +"); }
    public void volumeDown() { System.out.println("Radio vol -"); }
    public String name() { return "Radio"; }
}

// ----- Abstraction side (Remote hierarchy) -----
class RemoteControl {
    protected Device device; // THE BRIDGE
    RemoteControl(Device d) { device = d; }
    void volumeUp() { device.volumeUp(); }
    void volumeDown() { device.volumeDown(); }
}
class AdvancedRemote extends RemoteControl {
    AdvancedRemote(Device d) { super(d); }
    void mute() { System.out.println(device.name() + " muted"); }
}
  
```

```
// ----- Mix and match freely -----
RemoteControl r = new RemoteControl(new TV());
r.volumeUp();                                     // TV vol +

AdvancedRemote ar = new AdvancedRemote(new Radio());
ar.volumeUp();                                     // Radio vol +
ar.mute();                                         // Radio muted
```

Key Rules

- The abstraction holds the Implementor by interface — injected via constructor.
- Both hierarchies grow independently: add a new Remote without touching Device.
- Real Java example: JDBC — `java.sql.Connection` (abstraction), `MySQL/PgDriver` (implementor).
- Bridge is designed upfront; Adapter is a retrofit fix.

Q&A;

Q1. What is the core idea of Bridge?

Separate 'what you do' (abstraction) from 'how it is done' (implementation) so both hierarchies can change without affecting each other.

Q2. How does Bridge prevent class explosion?

Instead of one subclass per combination, you have one hierarchy per dimension. M abstractions + N implementors = $M+N$ classes instead of $M \times N$.

Q3. Bridge vs Adapter?

Adapter is a retrofit — makes two incompatible existing interfaces work together. Bridge is designed upfront to allow two dimensions to vary independently.

Q4. How is the implementor provided?

Through constructor injection. The abstraction declares it needs a Device; the caller provides one — the same mechanism used by Dependency Injection frameworks.

Q5. Real Java library example?

JDBC: `Connection/Statement/ResultSet` are the abstraction; `MySQL`, `PostgreSQL`, `Oracle` drivers are implementations. Swap drivers without touching SQL code.

Q6. Bridge vs Strategy?

Structurally similar (both use interface composition). Strategy focuses on swapping algorithms at runtime. Bridge focuses on separating two independent dimensions of variation.

C

Composite Pattern

Treat individual objects and groups of objects through the same interface

Composite builds **tree structures** where every node — leaf or composite — responds to the same interface. The client calls the same method on a single file or an entire folder tree without any type checks.

Simplest Example: File System

Both **File** (leaf) and **Folder** (composite) implement `FileSystemItem`. Calling `getSize()` on a `Folder` automatically recurses into all children.

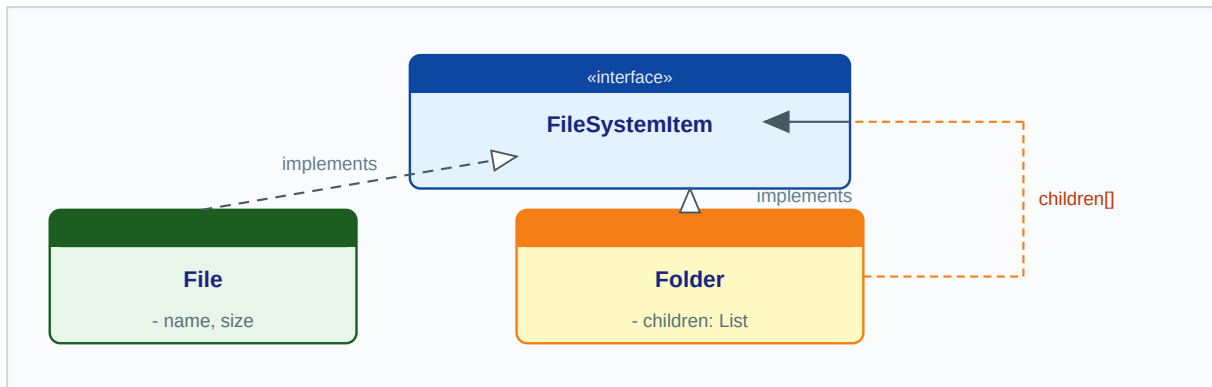


Fig 3 — Composite structure: File and Folder both implement `FileSystemItem`; Folder holds `children[]`

```

// Component – shared interface for leaf and composite
interface FileSystemItem {
    String getName();
    int    getSize();
    void   print(String indent);
}

// Leaf – no children, does real work
class File implements FileSystemItem {
    private String name; private int size;
    File(String n, int s) { name=n; size=s; }
    public String getName()      { return name; }
    public int    getSize()      { return size; }
    public void   print(String i) { System.out.println(i+name+" (" +size+"KB)"); }
}

// Composite – holds children, delegates to them
class Folder implements FileSystemItem {
    private String name;
    private List<FileSystemItem> children = new ArrayList<>();
    Folder(String n) { name=n; }
    void add(FileSystemItem e) { children.add(e); }
    public String getName()    { return name; }
    public int getSize() {
        return children.stream().mapToInt(FileSystemItem::getSize).sum();
    }
    public void print(String i) {
        System.out.println(i+"["+name+"]");
        children.forEach(c -> c.print(i+" "));
    }
}

// Usage – client treats File and Folder identically
Folder root = new Folder("root");
root.add(new File("readme.txt", 2));
  
```

```
Folder src = new Folder("src");
src.add(new File("Main.java", 10));
src.add(new File("App.java", 8));
root.add(src);
root.print("");
// [root]
//  readme.txt (2KB)
//  [src]
//    Main.java (10KB)
//    App.java (8KB)
System.out.println(root.getSize()); // 20
```

Key Rules

- Every node (leaf and composite) implements the same Component interface.
- Composite delegates its operation to each child recursively — traversal is automatic.
- Client never uses instanceof — same method call works on any node in the tree.
- Real Java examples: Swing JPanel/JButton hierarchy, XML DOM, java.awt.Component.

Q&A;

Q1. What are the three roles in Composite?

Component (shared interface), Leaf (no children, does real work), Composite (has children, delegates and aggregates results).

Q2. What is recursive composition?

A Composite calls the same operation on each child. If a child is also a Composite, it recurses further — the full tree is traversed automatically.

Q3. Transparent vs Safe Composite?

Transparent puts add/remove in Component (uniform API but Leaf must stub them). Safe puts add/remove only in Composite (Leaf is clean but client must cast to add children).

Q4. Real Java example?

Swing: JPanel.paint() calls paint() on every child recursively. JPanel is Composite; JButton, JLabel are Leaves.

Q5. Composite vs Decorator?

Composite is about tree structure (multiple children, aggregate results). Decorator wraps exactly one component to add behaviour. Composite is a tree; Decorator is a chain.

Q6. When to avoid Composite?

When the tree is not truly recursive (just one level), or when leaf and composite behaviour diverges so much that a common interface becomes forced.

D Decorator Pattern

Add responsibilities to an object dynamically by wrapping it

Decorator adds behaviour to individual objects **without subclassing**. Each decorator wraps a component, calls it, and adds its own work before or after. Wrap as many times as needed — the client sees only the top-level interface.

Simplest Example: Coffee with Milk and Sugar

Plain Coffee costs █10. Wrap it in **MilkDecorator** (+█2) or **SugarDecorator** (+█1). Stack them in any order. No subclass explosion, no changes to the original class.

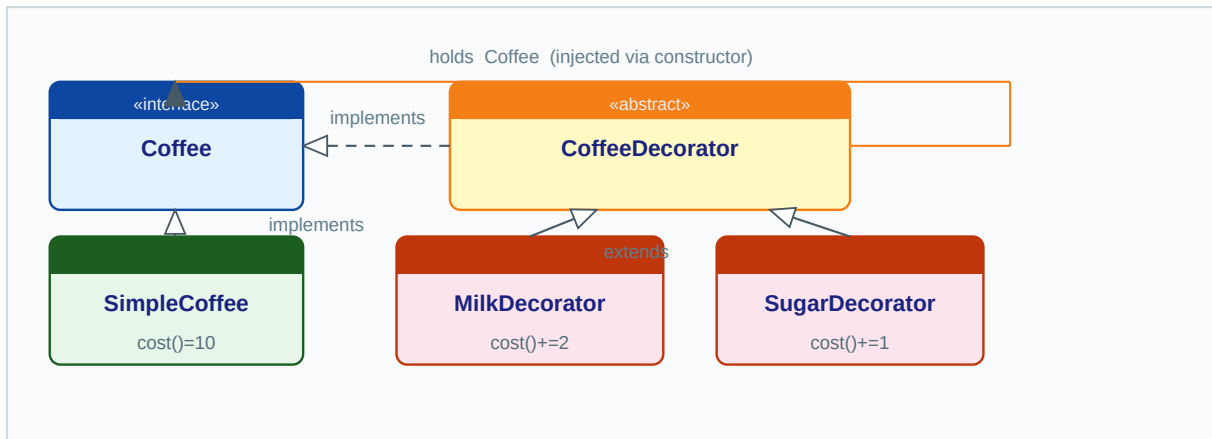


Fig 4 — Decorator structure: each decorator implements Coffee and holds a Coffee reference

```
// Component interface
interface Coffee {
    String description();
    int cost();
}

// Concrete Component
class SimpleCoffee implements Coffee {
    public String description() { return "Coffee"; }
    public int cost() { return 10; }
}

// Base Decorator – implements Coffee, holds a Coffee
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    CoffeeDecorator(Coffee c) { coffee = c; }
}

// Concrete Decorator 1
class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee c) { super(c); }
    public String description() { return coffee.description() + ", Milk"; }
    public int cost() { return coffee.cost() + 2; }
}

// Concrete Decorator 2
class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee c) { super(c); }
    public String description() { return coffee.description() + ", Sugar"; }
    public int cost() { return coffee.cost() + 1; }
}
```



```
// Usage – stack freely, in any order
Coffee c = new SimpleCoffee();           // Coffee = Rs 10
c = new MilkDecorator(c);                // Coffee, Milk = Rs 12
c = new SugarDecorator(c);               // Coffee, Milk, Sugar = Rs 13

Coffee fancy = new SugarDecorator(
    new MilkDecorator(
        new MilkDecorator(
            new SimpleCoffee())));
System.out.println(fancy.cost());        // 10+2+2+1 = 15
```

Key Rules

- Decorator implements the same interface as the component it wraps.
- It calls the wrapped component's method and adds its own behaviour around it.
- Stacking order matters when operations are not commutative (e.g. logging before caching).
- Real Java I/O example: `new BufferedReader(new InputStreamReader(new FileInputStream("f")))`.

Q&A;

Q1. What problem does Decorator solve?

Adding behaviour to individual objects without modifying their class or creating a subclass for every combination of features.

Q2. Decorator vs Inheritance?

Inheritance adds behaviour at compile time to all instances. Decorator adds at runtime to individual objects — and wrappers can be composed freely.

Q3. What must every Decorator do?

Implement the same interface as the component, hold a reference to the component, and call the component's method as part of its own operation.

Q4. Does order of decorators matter?

Yes when operations are not commutative. `LoggingDecorator(CachingDecorator(svc))` logs every call including cache hits; `CachingDecorator(LoggingDecorator(svc))` only logs cache misses.

Q5. Give a Java I/O example.

`new BufferedReader(new FileReader("f.txt"))` — `BufferedReader` wraps `FileReader` and adds buffering without changing `FileReader`'s interface.

Q6. How does Spring use Decorator?

`@Transactional` and `@Cacheable` proxies wrap your bean and add behaviour (transaction, cache) before/after delegating — structurally identical to Decorator.

Cheat Sheet — Structural Patterns Part 1

Pattern	Intent	Key roles	Java example	Spot the violation
Adapter	Translate incompatible interfaces	Target, Adaptee, Adapter	InputStreamReader, Arrays.asList	Client instantiates Adaptee directly
Bridge	Separate abstraction from impl	Abstraction, Implementor	JDBC (Connection + Driver)	Subclass per combination (TVRemote, RadioRemote...)
Composite	Treat leaf and group uniformly	Component, Leaf, Composite	Swing JPanel/JButton, DOM	Client uses instanceof to check leaf vs group
Decorator	Add behaviour by wrapping	Component, Decorator	BufferedReader, unmodifiableList	Subclass for every feature combination

How each pattern uses composition:

Adapter	Holds Adaptee	Translates calls to Adaptee API
Bridge	Holds Implementor	Delegates primitives to Implementor
Composite	Holds children[]	Delegates to each child recursively
Decorator	Holds Component	Calls Component then adds own behaviour

Key takeaway: All four patterns use composition over inheritance. Adapter *retrofits*, Bridge *anticipates*, Composite *builds trees*, Decorator *extends dynamically*.